

Министерство науки и высшего образования Российской Федерации
федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»

Университетский технологический колледж

наименование подразделения

Кафедра Информационные системы в экономике

наименование кафедры

Направление Информационные системы и программирование

Отчёт защищён с оценкой _____

_____ К.В Воробьев

(подпись руководителя от вуза) (инициалы, фамилия)

“ _____ ” _____ 2025г.

ОТЧЕТ

по лабораторной работе №4

Некоторые конструкции

тема лабораторной работы

по дисциплине Объектно-ориентированное программирование

ЛР 09.02.07.28.001 ПЗ

обозначение документа

Студент группы _____ 1ИСП-22

_____ М.С. Заковряшин

инициалы, фамилия

Руководитель работы _____ преподаватель

должность, ученое звание

_____ К.В. Воробьев

инициалы, фамилия

Барнаул 2025

Задачи:

1. Создайте базовый класс:

- Определите базовый класс с общими свойствами и методами, которые будут переопределяться в производных классах.
- Методы должны быть полиморфными, то есть их поведение должно зависеть от конкретного объекта.

```
class Figure(ABC):
    def __init__(self, color):
        self.color = self.color_validation(color)

    @abstractmethod
    def can_move(self, board, row, column, row1, column1):
        return True

    @abstractmethod
    def can_attack(self, board, row, column, row1, column1):
        return self.can_move(board, row, column, row1, column1)

    @abstractmethod
    def get_color(self):
        return self.color

    def color_validation(self, color):
        color = color.lower()
        if 'bl' in color or 'ck' in color:
            return 'black'
```

Рисунок 1 - базовый класс фигура

2. Создайте минимум два производных класса:

- Каждый производный класс должен иметь уникальные свойства и методы, расширяющие функциональность базового класса.
- Реализуйте полиморфные методы, которые будут работать по-разному в каждом из производных классов.

```

class Rook(Figure):
    def __init__(self, color):
        super().__init__(color)
        log_action(f"Creating {self}.")

    def can_move(self, board, row, column, row1, column1):
        return row == row1 or column == column1

    def can_attack(self, board, row, column, row1, column1):
        return self.can_move(board, row, column, row1, column1)

    def get_color(self):
        return super().get_color()

    def __str__(self):
        return f"Rook(color='{self.color}')"

    def __repr__(self):
        return f"Rook(color='{self.color}')"

```

Рисунок 2 - реализация полиморфных методов

3. Используйте лямбда-выражения:

- Создайте лямбда-функции для обработки объектов производных классов (например, для вывода информации, фильтрации или сортировки).

```

def sort_pieces(pieces):
    """Сортируем фигуры по типу"""
    return sorted(pieces, key=lambda piece: piece.__class__.__name__)

logging.basicConfig(filename='chess_game.log', level=logging.INFO,
                    format='%(asctime)s - %(levelname)s - %(message)s')

```

Рисунок 3 – сортировка

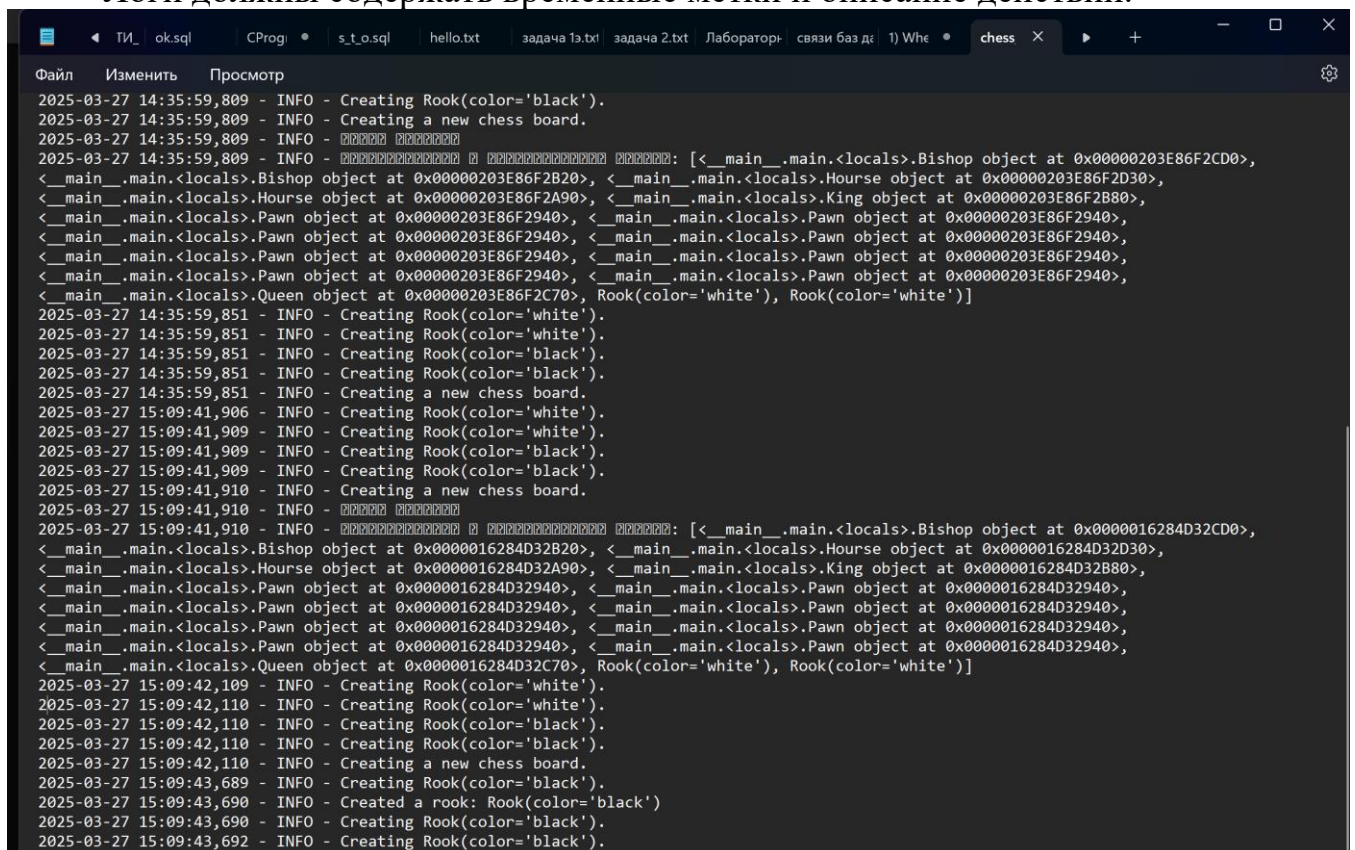
- Примените лямбда-выражения в контексте работы с коллекциями объектов (списки, словари и т.д.).

```
def create_board(self):
    for row in range(8):
        for col in range(8):
            color = "white" if (row + col) % 2 == 0 else "gray"
            button = tk.Button(self.root, bg=color, width=6, height=3,
                               command=lambda r=row, c=col: self.on_square_click(r, c))
            button.grid(row=row, column=col)
            self.buttons[row][col] = button
            self.update_square(row, col)
```

Рисунок 4 - использование лямбда функций для работы с массивом

4. Организуйте логирование:

- Создайте систему логирования, которая будет записывать ключевые действия программы (создание объектов, вызов методов и т.д.) в файл.
- Логи должны содержать временные метки и описание действий.



```
Файл  Изменить  Просмотр
2025-03-27 14:35:59,809 - INFO - Creating Rook(color='black').
2025-03-27 14:35:59,809 - INFO - Creating a new chess board.
2025-03-27 14:35:59,809 - INFO - 
2025-03-27 14:35:59,809 - INFO - 
[<__main__.main.<locals>.Bishop object at 0x00000203E86F2CD0>,
<__main__.main.<locals>.Bishop object at 0x00000203E86F2B20>, <__main__.main.<locals>.Hourse object at 0x00000203E86F2D30>,
<__main__.main.<locals>.Hourse object at 0x00000203E86F2A90>, <__main__.main.<locals>.King object at 0x00000203E86F2B80>,
<__main__.main.<locals>.Pawn object at 0x00000203E86F2940>, <__main__.main.<locals>.Pawn object at 0x00000203E86F2940>,
<__main__.main.<locals>.Pawn object at 0x00000203E86F2940>, <__main__.main.<locals>.Pawn object at 0x00000203E86F2940>,
<__main__.main.<locals>.Pawn object at 0x00000203E86F2940>, <__main__.main.<locals>.Pawn object at 0x00000203E86F2940>,
<__main__.main.<locals>.Queen object at 0x00000203E86F2C70>, Rook(color='white'), Rook(color='white')]
2025-03-27 14:35:59,851 - INFO - Creating Rook(color='white').
2025-03-27 14:35:59,851 - INFO - Creating Rook(color='white').
2025-03-27 14:35:59,851 - INFO - Creating Rook(color='black').
2025-03-27 14:35:59,851 - INFO - Creating Rook(color='black').
2025-03-27 14:35:59,851 - INFO - Creating a new chess board.
2025-03-27 15:09:41,906 - INFO - Creating Rook(color='white').
2025-03-27 15:09:41,909 - INFO - Creating Rook(color='white').
2025-03-27 15:09:41,909 - INFO - Creating Rook(color='black').
2025-03-27 15:09:41,909 - INFO - Creating Rook(color='black').
2025-03-27 15:09:41,910 - INFO - Creating a new chess board.
2025-03-27 15:09:41,910 - INFO - 
2025-03-27 15:09:41,910 - INFO - 
[<__main__.main.<locals>.Bishop object at 0x0000016284D32CD0>,
<__main__.main.<locals>.Bishop object at 0x0000016284D32B20>, <__main__.main.<locals>.Hourse object at 0x0000016284D3D30>,
<__main__.main.<locals>.Hourse object at 0x0000016284D32A90>, <__main__.main.<locals>.King object at 0x0000016284D32B80>,
<__main__.main.<locals>.Pawn object at 0x0000016284D32940>, <__main__.main.<locals>.Pawn object at 0x0000016284D32940>,
<__main__.main.<locals>.Pawn object at 0x0000016284D32940>, <__main__.main.<locals>.Pawn object at 0x0000016284D32940>,
<__main__.main.<locals>.Pawn object at 0x0000016284D32940>, <__main__.main.<locals>.Pawn object at 0x0000016284D32940>,
<__main__.main.<locals>.Queen object at 0x0000016284D32C70>, Rook(color='white'), Rook(color='white')]
2025-03-27 15:09:42,109 - INFO - Creating Rook(color='white').
2025-03-27 15:09:42,110 - INFO - Creating Rook(color='white').
2025-03-27 15:09:42,110 - INFO - Creating Rook(color='black').
2025-03-27 15:09:42,110 - INFO - Creating Rook(color='black').
2025-03-27 15:09:42,110 - INFO - Creating a new chess board.
2025-03-27 15:09:43,689 - INFO - Creating Rook(color='black').
2025-03-27 15:09:43,690 - INFO - Created a rook: Rook(color='black')
2025-03-27 15:09:43,690 - INFO - Creating Rook(color='black').
2025-03-27 15:09:43,692 - INFO - Creating Rook(color='black').
```

Рисунок 5 - файл логирования

5. Реализуйте графический интерфейс:

- Создайте графический интерфейс в Figma.

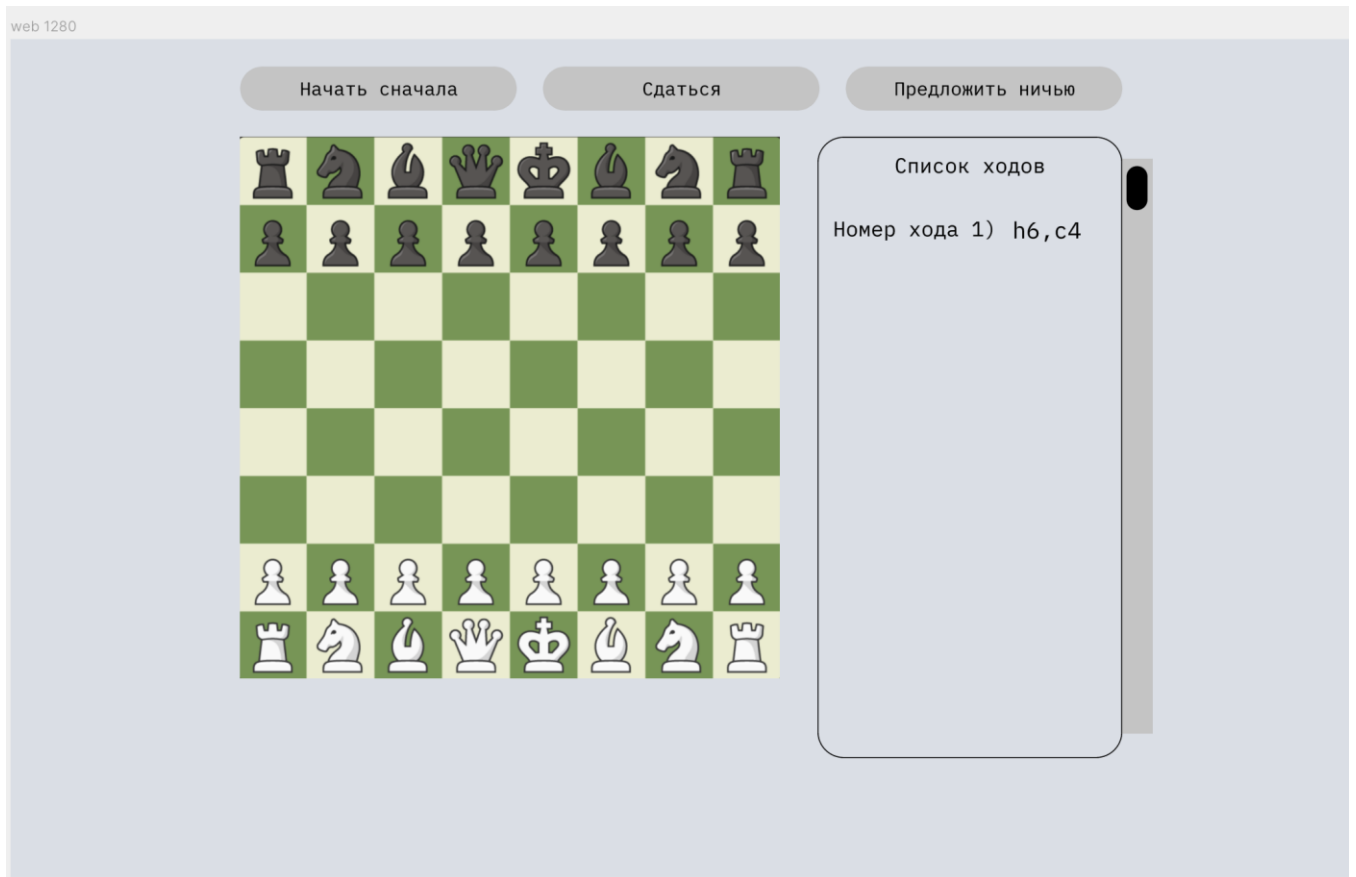


Рисунок 6 - интерфейс

Интерфейсы (опционально):

- Если ваша тема подразумевает использование интерфейсов, создайте абстрактный класс или интерфейс, который будет определять обязательные методы для производных классов.

https://github.com/LordFarqa/zakovryashin_1_isp_23_0/tree/object-oriented-programming